Temco Platform

Authoring parity with Dr.Explain, publishing parity with MSDN · built Nov 10, 2026

Authoring parity with Dr.Explain, publishing parity with MSDN

The product direction is two references at once, and they answer different questions:

- **Dr.Explain** is the *authoring* reference: how a person starts a project, brings existing material in, and edits topics with their properties in front of them.
- **MSDN** is the *output* reference: what a reader gets. The website is the product; CHM and PDF are the same tree in other clothes.

Where the two disagree, MSDN wins — a surface that looks like Dr.Explain but does not produce an MSDN-shaped site has failed the requirement.

1. Where we stand today

Everything below is what the code does, not what the mock draws.

Thing	Where	Reality today
A project	catalog `documents` row (`extensions/temco-store/src/catalog/schema.ts`)	name, slug, version, audience, owner, timestamps
What it reads	`doc_sources` (many-to-many, per-source branch + include glob)	created by the wizard's **Sources** step
What it builds	`recipes` (`engine`, `config_path`, `script`, `outputs`)	the wizard writes `engine: 'docfx'` and the ticked outputs
Outputs offered	`extensions/temco-projects/src/projectFlow.ts`	website · PDF · offline help (CHM) · search index · AI index
Creation flow	`src/projectFlow.ts`	4 steps: **Basics** (name, version, owner, audience) → **Sources** → **Outputs** → **Review**
Content	each product repository's `docs/help/registry.json` + Markdown (`extensions/temco-docs/src/registry.ts`)	the repository is the source of truth; the platform reads it
Page metadata already modelled	`RegistryPage`	`title, area, chapter, order, audience, ship, owner, tags, keywords, help_ids, code, reviewed`
Authoring already implemented	`newPagePage.ts` (3-step new page), `pageOps.ts` (`writePage` — the only disk write), `registryEdit.ts` (field writes)	a page is created from the location + template + draft, and registry fields survive regeneration
Builds	`extensions/temco-docs/src/builds.ts`	the **repository's own scripts** (`docs:site`, `docs:build`, `build:help:dry`) run as VS Code tasks, and success is judged by the artifacts
Checks	`diagnostics.ts`, `sections.ts`	structure, staleness, unreviewed pages

Two consequences worth stating plainly:

1. **We already have the Dr.Explain page properties.** `ship` is its "export conditions", `help_ids` is its "help context ID", `keywords` is its index, `order` is its TOC position, and `owner` / `audience` / `reviewed` are its bookkeeping. Nothing in the data model is missing for authoring.

2. **We are missing the front door and the import.** Creation starts at "name, version, owner" — a project is assumed to have content already. There is no way to begin from a Word file, an HTML folder, a CHM, a template, or a repository.

2. Dr.Explain's flow, from the surfaces

Start page:

- four ways in — **Create a local project**, **Create a shared project on the server**, **Checkout project from the server**, **Open an existing project**;
- six importers — **Microsoft Word**, **RTF**, **Compiled HTML Help (CHM)**, **HLP**, **Standalone HTML**, **folder of HTML/TXT/XML**;
- **recent projects** and **templates in all languages**.

Editor:

- topic tree with a synthetic *Title / Table of contents* root and numbered topics;
- **properties panel** per topic: title, filename, export conditions, status, header/footer, page numbering, help context IDs;
- central page with **Menu / Index / Search** tabs;
- toolbar previews: **Content editor**, **HTML preview**, **CHM preview**, **PDF preview**;
- AI assistant rail with the model/provider setting.

3. Gap analysis

Capability	Dr.Explain	Us today	What it means
Start from nothing	local project	**have it** (Basics step)	—
Start from a template	templates in all languages	no	a template is a repository-shaped skeleton; cheap and high value
Start from existing material	Word, RTF, CHM, HLP, HTML, folder	no	**the biggest gap**
Shared / checkout	server + checkout	git remote + clone exists in `temco-repositories`, not in the project flow	map to git, do not build a server
Topic tree editing	drag, renumber, nest	tree view + `order` field	add reorder → `order` write
Per-topic properties	full panel	registry fields, but no form	**form over `registryEdit`**
Export conditions	per topic	`ship`	expose it
Context IDs / index	per topic	`help_ids`, `keywords`	expose them
WYSIWYG body editing	HTML editor inside the tool	Markdown in the editor + rendered preview	do **not** copy; MSDN topics are Markdown
Preview before build	HTML · CHM · PDF	HTML preview exists; CHM/PDF only after a build	add CHM/PDF preview after first build
MSDN-shaped site	—	the site is the repository generator's; our output choice is `website`	make the contract explicit (§5)

4. Decisions

D1 — Creation asks *where the content comes from*, not a cosmetic "type". The origin decides which steps follow and what gets written on *Create*. A dropdown that changes nothing is worse than none.

Origin	What it does on Create	Writes
Blank project	creates the project; the first page is authored here	`documents` + `recipes`
From template	copies a template folder into the chosen location, then the project cites it	template tree + `documents` + `doc_sources`
From a repository (Dr.Explain's "shared")	clones/attaches a remote at a branch and reads its `docs/help/registry.json`	`sources` + `doc_sources`
Import a document set	converts the material into pages, then the project cites the folder	Markdown tree + registry entries + `doc_sources`
Open existing	picks a folder that already holds a registry	`sources` only

D2 — Importers write the repository's contract, never a second content store. An importer is a converter: source → `docs/help/*.md` + `docs/help/registry.json`, exactly the shape the product's own generator writes. The platform database stays what it is (a register of projects, sources, builds, releases) — otherwise the same text would live twice and drift.

D3 — MSDN is an output contract, not a theme. See §5.

D4 — Page properties are ours already; add only what MSDN needs. Expose `ship`, `order`, `keywords`, `help_ids`, `audience`, `owner` through a form in the page editor. Two new optional fields are worth having: `kind` (concept · how-to · reference · tutorial — MSDN's `ms.topic`) and `applies_to` (the product versions a page is for). Header/footer and page numbering are **template** settings for PDF, not per page.

D5 — Collaboration is git, not a server. "Shared project" = a project whose source is a repository with a remote; "checkout" = clone + branch; review is a pull request. `temco-repositories` already clones, pulls and records branches. Building a Dr.Explain server would duplicate what the repository host already does better.

D6 — The website is the product; CHM and PDF derive from it. One tree, three targets: website (primary), CHM (same topics, `help_ids` → HH context IDs, `keywords` → index), PDF (one print profile).

5. The MSDN contract (what "similar to MSDN" means concretely)

Structure. `area → chapter → topic`, the shape the registry already has. A topic is one Markdown file; `order` decides TOC position; a page not in the registry is not published.

URL. `/{area}/{chapter}/{slug}` — stable, lowercase, no build hash. Renaming a slug writes a redirect entry, because links outlive builds.

Every topic page carries:

- breadcrumb `Area / Chapter / Title`;
- left **TOC** of the whole set with the current branch open;
- an **on-page TOC** built from `##` / `###`;
- **Applies to** (from `applies_to`, or the project version);
- **previous / next** in TOC order;
- **feedback** and a "this page is stale" signal from `reviewed`;
- a permalink for every heading.

Index and search. `keywords` feeds the site index; the search index (already a wizard target) is built from titles, headings and keywords, so a reader can reach a page by a controller's name as well as by its title.

Version selector. A released version is a `releases` row; the site serves the newest plus archived versions. The version selector is a list of those rows, not a rebuild.

CHM. The same tree compiled with `help_ids` as context IDs, `keywords` as the index, and the chapters as the TOC levels. A topic that is `ship: false` is in the site's draft preview and not in the CHM.

Where the furniture is produced — and why Temco Learn is not it. A topic page is rendered by the product repository's own site generator (`docs:site`), and Learn's *Article Preview* opens the published file rather than drawing a second copy of it (`extensions/temco-workbench/src/learnPages.ts`: *"a preview that can disagree with the shipped page is worse than no preview"*). So the platform's half of the contract is **the metadata that makes the furniture possible** — `order` for the TOC, `keywords` for the index, `ship` for what is published, `help_ids` for the CHM, `title` for the heading and the breadcrumb — and the generator's half is drawing it. Adding a page-properties form (phase 2, below) is what closes the loop between them.

6. The project flow, redesigned

Five steps; step 2–4 are the steps we have, unchanged in content.

1. **Origin** — Blank · Template · Repository · Import · Open existing.
2. **Basics** — name, version, owner, audience (today's step 1).
3. **Bring in** — *shape follows the origin*: template picker · repository picker (remote + branch) · importer picker (Word/RTF/HTML/HTML folder/CHM) with a destination folder · folder picker.
4. **What it builds** — the targets we already offer, with **Website** ticked for a new project.
5. **Review** — what will be written, and where the files land. Nothing is written before this step.

Two rules stay: nothing is written until the last button, and the project you create becomes the selected project.

7. Phases

Phase	Deliverable	Status
1	`documents.origin` + `documents.template` (schema 4, with a migration); wizard step **Origin** (blank · repository · folder · template · import), where the origin decides what the third step asks; a template writes a starter `docs/help` tree and refuses to overwrite anything; folder and template projects record their folder as a source	**done**
2	Page-properties form (`temcoDocs.pageProperties` , Markdown title-bar action) over `registryEdit` : title, owner, audience, order, ships, keywords, tags, `help_ids` , with the confirmation naming every value it will write	**done**
3	Importers (`temcoDocs.import` , called by the wizard's *Import a document set* origin): **HTML folder / page** and **Markdown folder** natively; **Word / RTF / ODT** through pandoc; **CHM** through `hh.exe -decompile` (Windows). All four write the repository contract and refuse a folder that already has a registry	**done**
4	The platform's half of the MSDN contract: the metadata itself (`title` , `order` , `keywords` , `ship` , `help_ids` , `audience`), which phase 2 made editable and which the module's existing validation already reports on. A separate readiness page was built and then removed: it re-tested the same fields without adding a decision, and the Pipeline group already has Review Center and Build Center. The drawing half — breadcrumb, on-page TOC, prev/next, permalinks, redirects, version selector — belongs to each product repository's `docs:site` generator, which is not in this repository	**platform half done**
5	CHM/PDF previews after a build (`temcoDocs.openArtifact` , from the Build Center's artifact rows); drag reorder → `order` (already in `pageOps.handleDrop`); the repository origin cites recorded sources and links to the Sources page, where `temcoRepositories.fetchSource` clones	**done**

Phase 1 is the answer to "do we add more types?" — yes, five origins, and each one changes what the next step asks.

8. What we deliberately do not copy

- **Its WYSIWYG HTML editor.** MSDN topics are Markdown; the editor is VS Code's, and the page renders beside it.
- **Its project database.** The repository keeps the text; the platform keeps the register.
- **Its server and checkout.** Git and pull requests.
- **Its single-file exports as the main path.** The site is the product; CHM is one of its outputs.